TypeScript — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Setup

02 Basic Types

03 Functions and Parameters

04 Interfaces and Type Aliases

05 Union, Intersection and Literal Types

06 Classes and Access Modifiers

Intermediate (6 topics)

07 Type Narrowing and Guards

08 Generics

09 Enums and Const Assertions

10 Modules and Namespaces

11 tsconfig and Strict Mode

12 Working with JavaScript Libraries

Advanced (6 topics)

13 Utility Types

14 Mapped and Conditional Types

15 Template Literal and Index Types

16 Decorators

17 Discriminated Unions and Exhaustiveness

18 Async Types and Error Handling

Master (4 topics)

19	Advanced Type Patterns
20	TypeScript with React, Node and Angular
21	Testing and Linting
22	Large Codebases: Monorepos and Project References

Beginner Level

01. Introduction and Setup

TypeScript adds static types to JavaScript and compiles to plain JS. Install with `npm i -D typescript`, create `tsconfig.json` and compile with `tsc`.

```
npm i -D typescript
npx tsc --init
npx tsc
```

02. Basic Types

`string`, `number`, `boolean`, arrays, tuples, `enum`, `any`, `unknown`, `void`, `never`. Type inference means you often need no annotation.

```
let id: number = 1;
const tags: string[] = ['a'];
let pair: [string, number] = ['x', 1];
```

03. Functions and Parameters

Type parameters and return values; optional (`?`), default and rest parameters; function types.

```
function greet(name: string, greeting = 'Hello'): string {
  return `${greeting}, ${name}`;
}
```

04. Interfaces and Type Aliases

Describe object shapes with `interface` or `type`. Use `readonly` and optional properties.

```
interface User { readonly id: number; name: string; email?: string }
type ID = number | string;
```

05. Union, Intersection and Literal Types

`A | B` is either; `A & B` is both. Literal types restrict values to exact strings/numbers.

```
type Status = 'idle' | 'loading' | 'error';
type Admin = User & { role: 'admin' };
```

06. Classes and Access Modifiers

Classes with `public`, `private`, `protected`, `readonly`, parameter properties, `abstract` classes and `implements`.

```
class Account {
  constructor(private balance: number) {}
  deposit(n: number) { this.balance += n; }
}
```

Intermediate Level

07. Type Narrowing and Guards

Narrow a union using `typeof`, `instanceof`, `in`, equality and custom type guards (`x is T`).

```
function len(x: string | string[]) {
  return typeof x === 'string' ? x.length : x.length;
}
const isUser = (o: any): o is User => 'id' in o;
```

08. Generics

Write reusable code that works with many types while staying type-safe; constrain with `extends`.

```
function first<T>(items: T[]): T | undefined { return items[0]; }
function pick<T, K extends keyof T>(o: T, k: K): T[K] { return o[k]; }
```

09. Enums and Const Assertions

Prefer string unions or `as const` objects over `enum` in many cases.

```
const Roles = { Admin: 'admin', User: 'user' } as const;
type Role = typeof Roles[keyof typeof Roles];
```

10. Modules and Namespaces

Use ES `import/export`; `import type` imports types only. Configure `module` and `moduleResolution` in tsconfig.

```
import type { User } from './types';
export const x = 1;
```

11. tsconfig and Strict Mode

Enable `"strict": true`. Key options: `target`, `lib`, `noImplicitAny`, `strictNullChecks`, `paths`, `outDir`.

```
{ "compilerOptions": { "strict": true, "target": "ES2022", "module": "ESNext" } }
```

12. Working with JavaScript Libraries

Install types with `@types/package` or write `.d.ts` declaration files for untyped libraries.

```
npm i -D @types/node @types/express
```

Advanced Level

13. Utility Types

Built-ins: `Partial`, `Required`, `Readonly`, `Pick`, `Omit`, `Record`, `ReturnType`, `Parameters`, `Awaited`, `NonNullable`.

```
type UserPreview = Pick<User, 'id' | 'name'>;
type Draft = Partial<User>;
```

14. Mapped and Conditional Types

Transform types programmatically with `{ [K in keyof T]: ... }` and `T extends U ? X : Y`.

```
type Nullable<T> = { [K in keyof T]: T[K] | null };
type ElementOf<T> = T extends (infer E)[] ? E : never;
```

15. Template Literal and Index Types

Build string types: `` `${A}-${B}` ``; use `keyof`, `typeof` and indexed access `T['key']`.

```
type Event = `on${Capitalize<'click' | 'hover'>}`; // 'onClick' | 'onHover'
```

16. Decorators

Decorators add metadata or behaviour to classes and members; used heavily by Angular and NestJS.

```
function Log(target: any, key: string, d: PropertyDescriptor) {
  const f = d.value;
  d.value = (...a: any[]) => { console.log(key, a); return f(...a); };
}
```

17. Discriminated Unions and Exhaustiveness

Add a literal `kind` tag to each union member, then `switch` safely; use `never` to force handling all cases.

```
type Shape = { kind: 'circle'; r: number } | { kind: 'square'; s: number };
function area(s: Shape) { switch (s.kind) { case 'circle': return Math.PI * s.r ** 2; case
'square': return s.s ** 2; } }
```

18. Async Types and Error Handling

Type promises as `Promise<T>`; model results with union types (`Result<T,E>`) rather than throwing everywhere.

```
type Result<T> = { ok: true; value: T } | { ok: false; error: string };
async function load(): Promise<Result<User>> { /* ... */ }
```

Master Level

19. Advanced Type Patterns

Branded types, builder patterns, recursive types, `satisfies` and variadic tuples for very safe APIs.

```
type UserId = string & { __brand: 'UserId' };
const cfg = { port: 3000 } satisfies Record<string, number>;
```

20. TypeScript with React, Node and Angular

React: type props and hooks; Node: `@types/node` + `tsx` / `ts-node`; Angular: TypeScript is built in with strict templates.

```
type Props = { title: string };
const Card = ({ title }: Props) => <h1>{title}</h1>;
```

21. Testing and Linting

Test with Vitest/Jest + `ts-jest`; lint with `typescript-eslint`; use `tsc --noEmit` in CI.

```
npx tsc --noEmit
npx eslint .
```

22. Large Codebases: Monorepos and Project References

Use workspaces, `paths` aliases and `composite` project references for fast incremental builds.

```
{ "references": [{ "path": "../shared" }] }
```

Interview Question Bank

Q1. TypeScript vs JavaScript?
TypeScript is a typed superset that compiles to JavaScript, catching errors at compile time.

Q2. interface vs type?
Interfaces can be merged and extended; types can also express unions, tuples and mapped types.

Q3. any vs unknown vs never?
`any` disables checks, `unknown` requires narrowing, `never` is a value that cannot occur.

Q4. What are generics?
Type parameters that make functions and classes reusable and type-safe.

Q5. What is `strictNullChecks`?
Makes `null` and `undefined` separate types so you must handle them.

Q6. What are utility types?
Built-in generic helpers such as `Partial`, `Pick`, `Omit`, `Record`.

Q7. What is type narrowing?
Refining a broad type to a specific one with checks like `typeof` or `in`.

Q8. What is a discriminated union?
A union whose members share a literal tag property that enables safe `switch` narrowing.

Q9. Enum vs union of literals?
Unions emit no runtime code and are simpler; enums create an object at runtime.

Q10. What does `as const` do?
Makes values readonly and narrows them to literal types.

Capstone Project Ideas

1. Typed CLI to-do tool
2. Typed REST API client library
3. Generic data table component
4. State machine with discriminated unions
5. Type-safe event emitter